Chapter 14: Transactions

Chapter 14: Transactions

- Transaction Concept
- Transaction State
- Concurrent Executions
- Serializability
- Recoverability
- Implementation of Isolation
- Transaction Definition in SQL
- Testing for Serializability.

Transactions

- Collections of operations that form a single logical unit of work are called transactions.
  - A database system must ensure proper execution of transactions despite failures
  - Either the entire transaction executes, or none of it does.
  - Furthermore, it must manage concurrent execution of transactions in a way that avoids the introduction of inconsistency.

Transaction Concept

- A **transaction** is a *unit* of program execution that accesses and possibly updates various data items.
  - E.g. transaction to transfer $50 from account A to account B:

1. **read**(*A*)
2. *A* := *A* – 50
3. **write**(*A*)
4. **read**(*B*)
5. *B* := *B* + 50
6. **write**(*B*)

- Two main issues to deal with:
  - Failures of various kinds, such as hardware failures and system crashes
  - Concurrent execution of multiple transactions

Example of Fund Transfer

- Transaction to transfer $50 from account A to account B:
  1. **read**(*A*)
  2. *A* := *A* – 50
  3. **write**(*A*)
  4. **read**(*B*)
  5. *B* := *B* + 50
  6. **write**(*B*)
- **Atomicity requirement**
  - If the transaction fails after step 3 and before step 6, money will be “lost” leading to an inconsistent database state
    - Failure could be due to software or hardware
  - The system should ensure that updates of a partially executed transaction are not reflected in the database
- **Durability requirement**
  - Once the user has been notified that the transaction has completed (i.e., the transfer of the $50 has taken place), the updates to the database by the transaction must persist even if there are software or hardware failures.

Example of Fund Transfer (Cont.)

- Transaction to transfer $50 from account A to account B:
  1. **read**(*A*)
  2. *A* := *A* – 50
  3. **write**(*A*)
  4. **read**(*B*)
  5. *B* := *B* + 50
  6. **write**(*B*)
- **Consistency requirement** in above example:
  - the sum of A and B is unchanged by the execution of the transaction
  - In general, consistency requirements include
    - Explicitly specified integrity constraints such as primary keys and foreign keys
    - Implicit integrity constraints
      - e.g. sum of balances of all accounts, minus sum of loan amounts must equal value of cash-in-hand
  - A transaction must see a consistent database.
  - During transaction execution the database may be temporarily inconsistent.
  - When the transaction completes successfully the database must be consistent
    - Erroneous transaction logic can lead to inconsistency

Example of Fund Transfer (Cont.)

- **Isolation requirement** — if between steps 3 and 6, another transaction T2 is allowed to access the partially updated database, it will see an inconsistent database (the sum $A + B$ will be less than it should be).

T1	T2
1. **read**(A)	
2. $A := A - 50$	
3. **write**(A)	
	read(A), read(B), print(A+B)
4. **read**(B)	
5. $B := B + 50$	
6. **write**(B	

- Isolation can be ensured trivially by running transactions **serially**
  - that is, one after the other.
- However, executing multiple transactions concurrently has significant benefits, as we will see later.

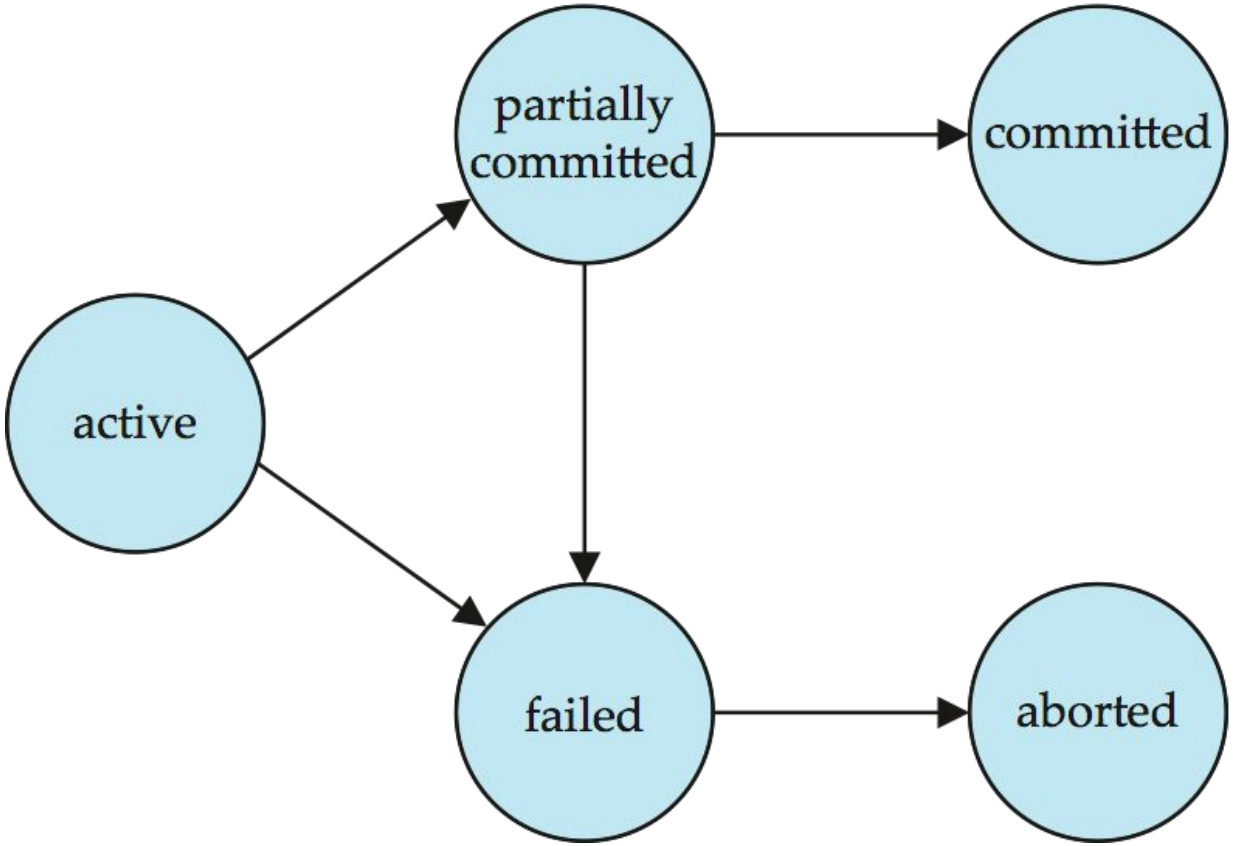
ACID Properties

- To preserve the integrity of data the database system must ensure:
  - **Atomicity**
    - Either all operations of the transaction are properly reflected in the database or none are.
  - **Consistency**
    - Execution of a transaction in isolation preserves the consistency of the database.
  - **Isolation**
    - Although multiple transactions may execute concurrently, each transaction must be unaware of other concurrently executing transactions
      - i.e. for every pair of transactions T_i and T_j, it appears to T_i that either T_j, finished execution before T_i started, or T_j started execution after T_i finished.
  - **Durability**
    - After a transaction completes successfully, the changes it has made to the database persist, even if there are system failures.

Transaction State

- **Active** – the initial state; the transaction stays in this state while it is executing
- **Partially committed** – after the final statement has been executed.
- **Failed** -- after the discovery that normal execution can no longer proceed.
- **Aborted** – after the transaction has been rolled back and the database restored to its state prior to the start of the transaction. Two options after it has been aborted:
  - restart the transaction
    - can be done only if no internal logical error
  - kill the transaction
- **Committed** – after successful completion.

Transaction State (Cont.)



Concurrent Executions

- Multiple transactions are allowed to run concurrently in the system. Advantages are:
  - **increased processor and disk utilization**, leading to better transaction *throughput*
    - E.g. one transaction can be using the CPU while another is reading from or writing to the disk
  - **reduced average response time** for transactions: short transactions need not wait behind long ones.
- **Concurrency control schemes** – mechanisms to achieve isolation
  - that is, to control the interaction among the concurrent transactions in order to prevent them from destroying the consistency of the database

Schedules

- **Schedule** – a sequences of instructions that specify the chronological order in which instructions of concurrent transactions are executed
  - A schedule for a set of transactions must consist of all instructions of those transactions
  - Must preserve the order in which the instructions appear in each individual transaction.
- A transaction that successfully completes its execution will have a commit instructions as the last statement
  - By default transaction assumed to execute commit instruction as its last step
- A transaction that fails to successfully complete its execution will have an abort instruction as the last statement

Schedule 1

- Let T_1 transfer \$50 from *A* to *B*, and T_2 transfer 10% of the balance from *A* to *B.*
- **A serial schedule in which T_1 is followed by T_2 :**

T_1	T_2
read (A)	
$A := A - 50$	
write (A)	
read (B)	
$B := B + 50$	
write (B)	
commit	
	read (A)
	$temp := A * 0.1$
	$A := A - temp$
	write (A)
	read (B)
	$B := B + temp$
	write (B)
	commit

- Suppose *A = $1000 and B = $2000,*
- Suppose also that the two transactions are executed one at a time in the order *T1 followed by T2.*
- *The final values of accounts A and B, after the* execution takes place, are \$855 and \$2145, respectively.
- Thus, the total amount of money in accounts *A and B—that is, the sumA + B—is preserved* after the execution of both transactions.

Schedule 2

- A serial schedule where T_2 is followed by T_1

T_1	T_2
	read (A) $temp := A * 0.1$ $A := A - temp$ write (A) read (B) $B := B + temp$ write (B) commit
read (A) $A := A - 50$ write (A) read (B) $B := B + 50$ write (B) commit	

if the transactions are executed one at a time in the order *T2 followed* by *T1* the sum *A + B is preserved, and the final values of accounts A and B* are $850 and $2150, respectively.

Schedule 3

- Let T_1 and T_2 be the transactions defined previously. The following schedule is not a serial (concurrent) schedule, but it is *equivalent* to Schedule 1

T_1	T_2
read (A)	
$A := A - 50$	
write (A)	
	read (A)
	$temp := A * 0.1$
	$A := A - temp$
	write (A)
read (B)	
$B := B + 50$	
write (B)	
commit	
	read (B)
	$B := B + temp$
	write (B)
	commit

If two transactions are running concurrently, the operating system may execute one transaction for a little while, then perform a context switch, execute the second transaction for some time, and then switch back to the first transaction for some time, and so on.
With multiple transactions, the CPU time is shared among all the transactions.

In Schedules 1, 2 and 3, the sum A + B is preserved.

Schedule 4

- The following concurrent schedule does not preserve the value of ($A + B$).

T_1	T_2
read (A)	
$A := A - 50$	
	read (A)
	$temp := A * 0.1$
	$A := A - temp$
	write (A)
	read (B)
write (A)	
read (B)	
$B := B + 50$	
write (B)	
commit	
	$B := B + temp$
	write (B)
	commit

Schedule 4—a concurrent schedule resulting in an inconsistent state.

- Not all concurrent executions result in a correct state
- After the execution of this schedule, we arrive at a state where the final values of accounts *A and B are $950 and $2100, respectively.*
- This final state is an *inconsistent state, since we have gained $50 in the process of* the concurrent execution
- Indeed, the sum *A + B is not preserved by the execution* of the two transactions.
- If control of concurrent execution is left entirely to the operating system, many possible schedules, including ones that leave the database in an inconsistent state, are possible.
- It is the job of the database system to ensure that any schedule that is executed will leave the database in a consistent state.
- The **concurrency-control** component of the database system carries out this task.

Serializability

- **Basic Assumption** – Each transaction preserves database consistency.
- Thus serial execution of a set of transactions preserves database consistency.
- A (possibly concurrent) schedule is serializable if it is equivalent to a serial schedule. Different forms of schedule equivalence give rise to the notions of:
  1. **Conflict serializability**
  2. **View serializability**

Simplified view of transactions

- We ignore operations other than **read** and **write** instructions
- We assume that transactions may perform arbitrary computations on data in local buffers in between reads and writes.
- Our simplified schedules consist of only **read** and **write** instructions.

Conflicting Instructions

- Let us consider a schedule *S in which there are two consecutive instructions, I and J , of transactions Ti and Tj , respectively (i = j). If I and J refer to different* data items, then we can swap *I and J without affecting the results of any instruction* in the schedule. However, if *I and J refer to the same data item Q, then the* order of the two steps may matter. Since we are dealing with only read and write instructions, there are four cases that we need to consider:
- **1. *I = read(Q), J = read(Q).***
  - ***The order of I and J does not matter, since the*** same value of *Q is read by Ti and Tj , regardless of the order.*
- **2. *I = read(Q), J = write(Q).***
  - ***If I comes before J , then Ti does not read the value*** of *Q that is written by Tj in instruction J . If J comes before I, then Ti reads* the value of *Q that is written by Tj. Thus, the order of I and J matters.*

Conflict Serializability

- 3. I = write(Q), J = read(Q). The order of I and J matters for reasons similar to those of the previous case.
- 4. I = write(Q), J = write(Q). Since both instructions are write operations, the order of these instructions does not affect either Ti or Tj .
- However, the value obtained by the next read(Q) instruction of S is affected, since the result of only the latter of the two write instructions is preserved in the database
- If there is no other write(Q) instruction after I and J in S, then the order of I and J directly affects the final value of Q in the database state that results from schedule S.

NOTE: We say that I and J conflict if they are operations by different transactions on the same data item, and at least one of these instructions is a write operation.

Conflict Serializability (Cont.)

- If a schedule *S* can be transformed into a schedule *S´* by a series of swaps of non-conflicting instructions, we say that *S* and *S´* are **conflict equivalent**.
- We say that a schedule *S* is **conflict serializable** if it is conflict equivalent to a serial schedule

Conflict Serializability (Cont.)

- Schedule 3 can be transformed into Schedule 6, a serial schedule where T_2 follows T_1, by series of swaps of non-conflicting instructions. Therefore Schedule 3 is **conflict serializable**.

T_1	T_2
read (A)	
write (A)	
	read (A)
	write (A)
read (B)	
write (B)	
	read (B)
	write (B)

Schedule 3

T_1	T_2
read (A)	
write (A)	
read (B)	
write (B)	
	read (A)
	write (A)
	read (B)
	write (B)

Schedule 6

1. Swap the read(B) *instruction of T1 with the write(A) instruction of T2.*
2. Swap the read(*B) instruction of T1 with the read(A) instruction of T2.*
3. Swap the write(*B) instruction of T1 with the write(A) instruction of T2.*
4. Swap the write(*B) instruction of T1 with the read(A) instruction of T2.*

Conflict Serializability (Cont.)

- Example of a schedule that is not conflict serializable:

T_3	T_4
read (Q)	
	write (Q)
write (Q)	

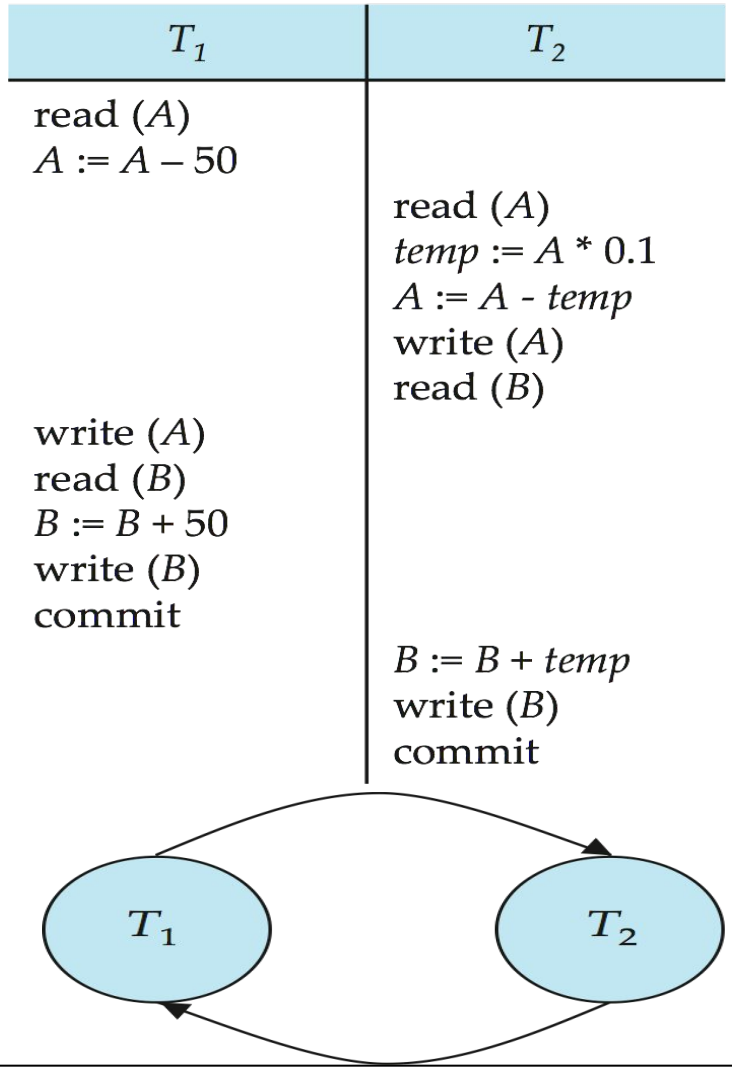
- We are unable to swap instructions in the above schedule to obtain either the serial schedule < T_3, T_4 >, or the serial schedule < T_4, T_3 >.

Testing for Serializability

- Method for determining conflict serializability of a schedule
- Consider a schedule S
  - We construct a directed graph, called a precedence graph, from S
  - This graph consists of a pair G = (V, E), where V is a set of vertices and E is a set of edges
  - The set of vertices consists of all the transactions participating in the schedule. The set of edges consists of all edges Ti →Tj for which one of three conditions holds:
    1. Ti executes write(Q) before Tj executes read(Q).
    2. Ti executes read(Q) before Tj executes write(Q).
    3. Ti executes write(Q) before Tj executes write(Q).

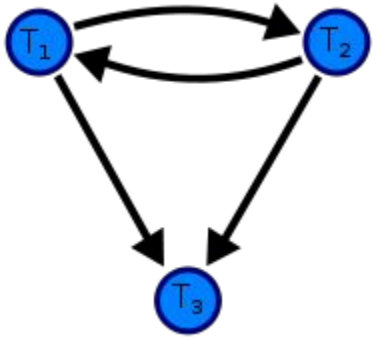
If an edge Ti → Tj exists in the precedence graph, then, in any serial schedule S' equivalent to S, Ti must appear before Tj .

- If the precedence graph for S has a cycle, then schedule S is not conflict serializable.
- If the graph contains no cycles, then the schedule S is conflict serializable
- The precedence graph for schedule 4 appears in the figure
- It contains the edge T1 →T2, because T1 executes read(A) before T2 executes write(A)
- It also contains the edge T2→T1, because T2 executes read(B) before T1 executes write(B).
- Thus, to test for conflict serializability, we need to construct the precedence graph and to invoke a cycle-detection algorithm.



The precedence graph contains a cycle, indicating that this schedule is not conflict serializable.

$$D = \begin{bmatrix} T1 & T2 & T3 \\ R(A) & R(B) & \\ & W(A) & \\ & & \\ W(A) & & \\ & & W(B) \\ & & W(A) \\ & & \end{bmatrix}$$



The drawing sequence for the precedence graph:-

1. For each transaction T_i participating in schedule S, create a node labelled T_i in the precedence graph. So the precedence graph contains T_1, T_2, T_3
2. For each case in S where T_i executes a write_item(X) then T_j executes a read_item(X), create an edge (T_i --> T_j) in the precedence graph. This occurs nowhere in the above example, as there is no read after write.
3. For each case in S where T_i executes a read_item(X) then T_j executes a write_item(X), create an edge (T_i --> T_j) in the precedence graph. This will bring to front a directed graph from T_1 to T_2.
4. For each case in S where T_i executes a write_item(X) then T_j executes a write_item(X), create an edge (T_i --> T_j) in the precedence graph. It creates a directed graph from T_2 to T_1, T_1 to T_3, and T_2 to T_3.
5. The schedule S is serializable if the precedence graph has no cycles. As T_1 and T_2 constitute a cycle, then we cannot declare S as serializable or not and serializability has to be checked using other methods.

Transaction Isolation and Atomicity

- If a transaction Ti fails, for whatever reason, we need to undo the effect of this transaction to ensure the atomicity property of the transaction.
- In a system that allows concurrent execution, the atomicity property requires that any transaction Tj that is dependent on Ti (that is, Tj has read data written by Ti) is also aborted.
- To achieve this, we need to place restrictions on the type of schedules permitted in the system.

Recoverable Schedules

Need to address the effect of transaction failures on concurrently running transactions.

- **Recoverable schedule** — if a transaction T_j reads a data item previously written by a transaction T_i, then the commit operation of T_i appears before the commit operation of T_j.
- The following schedule is not recoverable if T_9 commits immediately after the read
- T8 is called a **partial schedule**

because **commit** is missing

T_8	T_9
read (A)	
write (A)	
	read (A)
	commit
read (B)	

- If T_8 should abort, T_9 would have read (and possibly shown to the user) an inconsistent database state. Hence, database must ensure that schedules are recoverable
- To make it recoverable, T9 would have to delay committing until after T8 commits.

Cascading Rollbacks

- **Cascading rollback** – a single transaction failure leads to a series of transaction rollbacks. Consider the following schedule where none of the transactions has yet committed (so the schedule is recoverable)

T_{10}	T_{11}	T_{12}
read (A)		
read (B)		
write (A)		
	read (A)	
	write (A)	
		read (A)
abort		

If T_{10} fails, T_{11} and T_{12} must also be rolled back.

- Cascading rollback is undesirable, since it leads to the undoing of a significant amount of work.

Transaction T10 writes a value of A that is read by transaction T11.
Transaction T11 writes a value of A that is read by transaction T12.
Suppose that, at this point, T10 fails. T10 must be rolled back.
Since T11 is dependent on T10, T11 must be rolled back.
Since T12 is dependent on T11, T12 must be rolled back.

Cascadeless Schedules

- **Cascadeless schedules** — cascading rollbacks cannot occur; for each pair of transactions T_i and T_j such that T_j reads a data item previously written by T_i, the commit operation of T_i appears before the read operation of T_j.
- Every cascadeless schedule is also recoverable
- It is desirable to restrict the schedules to those that are cascadeless

Concurrency Control

- A database must provide a mechanism that will ensure that all possible schedules are
  - either conflict or view serializable, and
  - are recoverable and preferably cascadeless
- A policy in which only one transaction can execute at a time generates serial schedules, but provides a poor degree of concurrency
  - Are serial schedules recoverable/cascadeless?
- Testing a schedule for serializability *after* it has executed is a little too late!
- **Goal** – to develop concurrency control protocols that will assure serializability.

Concurrency Control vs. Serializability Tests

- Concurrency-control protocols allow concurrent schedules, but ensure that the schedules are conflict/view serializable, and are recoverable and cascadeless .
- Concurrency control protocols generally do not examine the precedence graph as it is being created
  - Instead a protocol imposes a discipline that avoids nonseralizable schedules.
  - We study such protocols in Chapter 16.
- Different concurrency control protocols provide different tradeoffs between the amount of concurrency they allow and the amount of overhead that they incur.
- Tests for serializability help us understand why a concurrency control protocol is correct.

Weak Levels of Consistency

- Some applications are willing to live with weak levels of consistency, allowing schedules that are not serializable
  - E.g. a read-only transaction that wants to get an approximate total balance of all accounts
  - E.g. database statistics computed for query optimization can be approximate (why?)
  - Such transactions need not be serializable with respect to other transactions
- Tradeoff accuracy for performance

End of Chapter 14